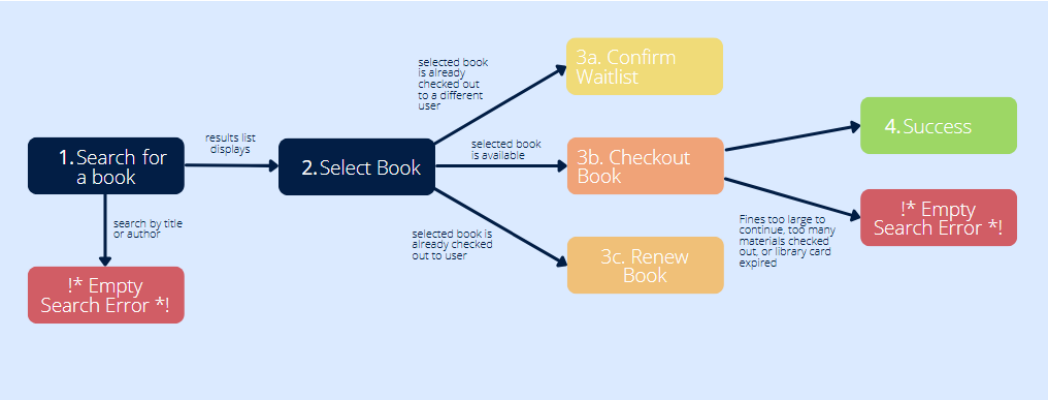
Case Study 02: Interactive Prototype Specification

1. Interaction Mechanics & Input Paradigm Mapping

- **User Flow Selection**:
    - Library website where users can search for and find a desired book, check it out if available, waitlist themselves for it if it is unavailable, or renew it if it is already checked out to them.



- **Input Mechanics Analysis**:

Input Type	Where it is Used	How it is Supported
Mouse (or other pointing device)	All buttons, fields, search inputs, dropdowns, checkboxes	Hover states, large easy to hit click targets, clearly contrasting elements
Touch	Same as above	Same as above
Keyboard	The entire workflow	Clicking tab moves through the page in a logical, chronological order, enter presses buttons, buttons are spaced out an easy to select
Screen Reader	The entire workflow	All fields are labeled, buttons for screen readers include titles being checked out/waitlisted/renewed.

		Search results can be easily read.

- **Fitts' Law Target Sizing**:
    - Audit all primary action elements (buttons, form controls, link targets) to ensure touch targets meet minimum ergonomic sizes (≥44x44px or 48x48px) and are spaced appropriately to prevent accidental input errors.

Element	Size	Spacing	Result
Buttons (search, checkout, waitlist, renew, confirm, back)	Minimum 48 x 48 px	12px	Pass
Search bar	48px tall	12px	Pass
Confirm/Agree checkbox	48 px tall	n/a	Pass
Borrowing rules toggle	48 px tall	n/a	pass

2. High-Fidelity Prototyping & Micro-Interaction Design

State	Visual	Behind the Scenes
Default	Blue buttons, fields with a grey border	Normal button
Hover	Buttons and or field bordered become a darker shade of their original color	CSS :hover
Focus	Thick black outline	CSS :focus-visible
Active/clicked	Button turns to its darkest possible shade	CSS :active
Disabled	Grey button with a dashed border, text explaining that the button is unavailable and why (ex. “Confirm button is unavailable until you agree to the rules”	aria-disabled="true" + aria-describedby (stays focusable so the reason can be read)

Loading	Loading spinner and text such as “Searching…”	aria-busy="true", polite announcement, busy flag blocks repeat clicks. Fake delays 1 to 1.5s
Error	Red border, pink background, text such as “Type a title or author to search, for example Dune” or “$4.50 Fines Owed”	aria-invalid="true", aria-live="assertive", role="alert", focus moved to message

3. Accessibility Architecture & WCAG Specification

- **Semantic HTML Markup**:
    - HTML5 layout structure:
    - (<header> with <nav aria-label=”Progress”> (aria changes to march current step),
    - <main> containing one clearly labeled <section> per screen (Find, Confirm, Done)
    - <form>, <fieldset> with <legend>, and a <label> on every control
    - Headings: h1 for site name, h2 for each step, h3 for results

- **ARIA & Screen Reader Hooks:**

Need	**Implementation**
Collapsible borrowing rules	aria-expanded="true/false" with aria-controls
User feedback and status updates (3 books found, renewal complete, etc.)	aria-live="polite" (role="status")
Inline form errors	aria-live="assertive" message + aria-invalid on the field
Confirm failure (fines, expired card, too many materials checked out)	role="alert" (assertive) box that receives focus
Keyboard tab order	Matches page order (search box, filters, search button, results, rules toggle, agree checkbox, back, confirm)

- **Visual Accessibility:**

What was Checked	Score	Needed
Main text on white	17.4	4.5
Hint text on white	8.27	4.5
White text on blue button	6.39	4.5
Greyed out button text	6.86	4.5
Red error text on white	6.54	4.5
Field border on white	4.93	3
Black focus outline on white	21	3

4. Technical Specification Deck Compilation

- Prototype Link: https://ashlynrv.github.io/AshlynRV-Case-Study-02-Library-Interactive-Prototype-Specification/
- Test card numbers:
    - 000000 has fines
    - 111111 is expired
    - 222222 has too many items checked out
    - 123456 works with no errors

5. Post-Mortem Reflection (Metacognition)

1. *How did explicitly defining every component state (Hover, Focus, Active, Loading, Error) during the specification phase prevent dynamic usability bugs that static wireframes completely fail to address?*

I am not always the biggest planner. I would never write drafts of my essays or draw prototypes for designs. I would always just create one thing and then revise and revise and revise it. Defining every component state before starting forced me to take a step back and make a plan. This helped me realize things like that the confirm button should be greyed out until a user agrees to the rules. Planning it out step by step like this also made me realize that I had to make the confirm button still focusable so that a screen reader can see it and understand why it is unavailable.

2. *Describe a scenario in your prototype where system latency or asynchronous data processing required specialized micro-interaction feedback (e.g., loading skeletons or aria-live announcements). How does your design keep the user informed and prevent repeated submission errors?*

In a real library website, there would be a delay between when a user checks out a book and when this is communicated to the library's server. So I built in a loading spinner so users would know the system was working and not prematurely exit or repeatedly click. This also communicates to screen readers so that users who cannot see the loading spinner still know the system is working.